

Paragliding Intelligence Platform

Volume 14 - Hardware Weather Station & Sensor Network Architecture v2 Technical

Developer-ready architecture for launch-site sensors, LoRaWAN/LTE telemetry, station health, calibration, edge firmware, ingestion, APIs, and business operations.

Document status: Technical specification v2

Primary audience: product leadership, backend engineers, firmware engineers, DevOps, clubs, schools, and hardware partners

Relationship to previous volumes: extends Volume 03 database, Volume 04 APIs, Volume 05 weather engine, Volume 06 AI/ML, Volume 08 infrastructure, Volume 09 security, and Volume 13 launch-site playbooks.

Table of Contents

1. Executive Summary
2. Design Goals and Non-Goals
3. Station Product Lines
4. Sensor Suite and Measurement Requirements
5. Hardware Reference Architecture
6. Connectivity Architecture
7. Edge Firmware Architecture
8. Payloads and Protocols
9. Cloud Ingestion Architecture
10. Database Extensions
11. APIs and Service Contracts
12. Quality Control and Calibration
13. Station Health and Operations
14. Weather Engine Integration
15. Mobile and Web UX
16. Security and Privacy
17. Installation Playbook
18. Manufacturing and Supply Chain
19. Commercial Model
20. MVP Scope and Roadmap
21. Acceptance Criteria
22. Engineering Backlog

1. Executive Summary

This volume defines the hardware weather-station and sensor-network architecture for a paragliding-first weather intelligence platform. Generic forecast models are not enough for mountain flying: launch wind, gust spread, valley flow, rotor, cloudbase, and fast-changing microclimates must be validated by real observations at the site. The sensor network turns the app from a forecast viewer into a reality-aware decision system.

The station program has four strategic purposes: collect local launch and landing observations, improve flyability and risk scoring, train site-specific digital twins, and create a club/school hardware business line. It is designed to work in remote mountain locations using solar power, LoRaWAN where available, LTE-M/NB-IoT or standard cellular fallback where needed, and store-and-forward behavior when connectivity drops.

This architecture assumes that LoRaWAN is useful for low-power telemetry with adaptive data-rate behavior, MQTT is useful for streaming event ingestion, and cellular IoT networks are a viable fallback where LPWAN coverage is unavailable. The design avoids hard dependence on one vendor by using a protocol adapter layer.

1.1 Competitive Advantage

Problem in existing weather apps	Sensor-network answer
Forecast layers show modeled wind, not actual launch wind.	Install launch sensors and display forecast-versus-reality deltas.
Pilots do not know if a site is worth driving to.	Use recent station data, webcam state, and forecast confidence to compute Drive/Do-not-drive.
Mountain microclimates differ strongly from grid models.	Train digital twins using forecast error samples from each launch.
Community reports are useful but inconsistent.	Combine trusted pilot reports with instrument observations and quality controls.
Clubs lack a structured way to run local meteo assets.	Provide station dashboards, maintenance alerts, and club ownership workflows.

2. Design Goals and Non-Goals

2.1 Goals

- Capture site-level observations at launch, landing, and ridge/valley reference points.
- Operate reliably in remote mountain terrain with intermittent connectivity and limited power.
- Normalize all device data into a canonical observation model usable by weather, AI, risk, and mobile services.
- Support heterogeneous devices: first-party stations, third-party club stations, webcams, and future instrument partners.
- Provide station-health observability: battery, solar charge, uptime, signal quality, stale data, sensor drift, and enclosure status.
- Produce forecast correction samples for digital twin and ML training.
- Expose secure APIs for station owners, clubs, schools, and enterprise partners.

2.2 Non-Goals for MVP

- No safety-critical certification claim. The station network provides decision support, not flight authorization.
- No dependency on aviation-certified instruments for MVP.
- No global manufacturing operation in MVP; use validated off-the-shelf components and limited pilot deployment.
- No mandatory custom hardware for data ingestion; third-party weather stations can be connected through adapters.

3. Station Product Lines

The platform should not start with one universal station. Different launches have different constraints: some have road access and power; others require solar, snow load, and low-bandwidth telemetry. The architecture defines product tiers while keeping the cloud ingestion model uniform.

Station type	Target deployment	Sensors	Connectivity	Business use
Micro Launch Node	Small clubs, temporary sites, test deployments	Wind speed/direction, temperature, humidity, pressure, battery	LoRaWAN or LTE-M/NB-IoT	Low-cost site coverage and MVP pilots
Standard Launch Station	Main launch sites	Ultrasonic wind, gusts, temp/humidity/pressure, solar/battery, tilt, enclosure temp	LoRaWAN plus LTE fallback	Paid club station package
Landing/Valley Station	Landing zones and valley winds	Wind, gusts, temp/humidity/pressure, optional rain	LTE or LoRaWAN	Valley wind and landing safety
Premium Vision Station	High-traffic sites	Standard station plus webcam, visibility/cloud analysis, optional solar radiation	LTE/Ethernet/Wi-Fi	Pro+/Club differentiator
Portable Event Node	Competitions/events	Wind and weather telemetry, GPS, battery, rugged case	LTE and local Wi-Fi	Event package rental
Third-Party Adapter	Existing station owners	Depends on source device	HTTP, MQTT, FTP, vendor APIs	Fast network expansion

4. Sensor Suite and Measurement Requirements

4.1 Canonical Sensor Variables

Variable	Unit	Minimum frequency	Why paragliding needs it	Quality checks
wind_speed_avg	m/s	60 s	Launch safety and scoring	range, spike, stuck sensor
wind_gust_max	m/s	60 s	Collapse/launch danger	gust spread threshold
wind_direction	degrees	60 s	Launch orientation compatibility	wrap-around, vector averaging
temperature	C	5 min	thermal potential, density altitude	range, drift
relative_humidity	%	5 min	cloudbase proxy	range, dewpoint consistency
pressure	hPa	5 min	weather trend and elevation sanity	sea-level correction, drift
rain_rate	mm/h	1-5 min	hard no-go condition	bucket debounce/radar cross-check
solar_irradiance	W/m2	1-5 min	thermal trigger confidence	day/night sanity
battery_voltage	V	5 min	station health	low threshold, discharge curve
solar_current	A	5 min	power diagnostics	night sanity
rsi_snr	dB	uplink	radio health	coverage trend
camera_frame_status	enum	5-10 min	visibility and cloud observation	stale/blur/darkness

4.2 Wind Measurement Design

Paragliding launch decisions are strongly affected by gusts, direction shifts, and local acceleration. For permanent stations, ultrasonic anemometers are preferred because they have no moving parts and lower mechanical maintenance. For low-cost MVP deployments, mechanical cup/vane sensors are acceptable if the app labels the source type and confidence.

Decision	Recommended MVP	Recommended V1+
----------	-----------------	-----------------

Paragliding Intelligence Platform - Volume 14 - Hardware Weather Station & Sensor Network Architecture v2 Technical

Sensor type	Mechanical wind speed and vane acceptable for pilots/test sites	Ultrasonic wind sensor for core commercial stations
Averaging	1-minute vector average plus 10-minute summary	1-second sample, 1-minute and 10-minute aggregates
Gust	Maximum 3-second or fastest available rolling gust within 60 seconds	Standardized gust calculation in edge firmware and cloud
Direction	Vector average, not arithmetic average	Vector average plus variability/stddev
Mounting	Mast above immediate obstacles	Documented mast height, exposure class and photos

5. Hardware Reference Architecture

5.1 Station Block Diagram

```

[Sensors]
|-- wind sensor
|-- temp/humidity/pressure
|-- rain / solar radiation / camera optional
|
[MCU / Edge Controller]
|-- sensor drivers
|-- local QC and aggregation
|-- secure identity
|-- offline buffer
|
[Connectivity]
|-- LoRaWAN radio
|-- LTE-M/NB-IoT or LTE fallback
|-- Wi-Fi/Ethernet optional
|-- BLE commissioning
|
[Power]
|-- solar panel
|-- charge controller
|-- battery
|-- low-power sleep profile
|
[Cloud Ingestion]
|-- device adapter
|-- observation normalizer
|-- QC pipeline
|-- weather engine / digital twin

```

5.2 Hardware Modules

Module	Responsibilities	Implementation notes
Edge controller	Collect, aggregate, timestamp, validate, and transmit sensor readings	ESP32-class MCU for MVP; industrial controller for premium stations; watchdog timer required
GNSS or time source	Accurate station location/time and deployment validation	GNSS optional for permanent station; NTP via LTE/Wi-Fi otherwise
Sensor bus	Connect wind/weather sensors	I2C/SPI/UART/RS485 depending on sensor class; isolate long cable runs
Connectivity module	LoRaWAN and/or cellular uplink	Modular radio interface preferred; fallback path configured per site
Power subsystem	Solar charging and battery runtime	Size for winter, shade and snow; report battery and charge metrics
Enclosure	Weatherproofing and serviceability	IP-rated enclosure, breathable vent, cable glands, condensation management
Mounting kit	Safe and repeatable installation	Mast, clamps, grounding consideration, photos required in commissioning

5.3 Edge Compute Rules

- Every station must compute 1-minute local summaries so short connectivity outages do not destroy gust data.
- Every payload must include station_id, firmware_version, sequence_number, observed_at, transmitted_at, and measurement interval.
- Every station must preserve at least 24 hours of summaries in non-volatile local storage where hardware allows.
- Every station must support remote configuration for sample interval, transmit interval, calibration coefficients, and enabled sensors.
- Premium stations must support OTA firmware updates with signed artifacts and rollback.

6. Connectivity Architecture

6.1 Connectivity Selection Matrix

Connectivity	Best use	Pros	Risks / limits	Cloud path
LoRaWAN	Remote low-power telemetry where gateway coverage exists	Low power, long range, good for small weather payloads	Payload size and downlink limits; coverage depends on gateways	Network server -> MQTT/Webhook -> station-ingest
LTE-M / NB-IoT	Remote stations without LoRaWAN or with commercial-grade uptime needs	Licensed spectrum, broad IoT use, good fallback	Operator coverage/roaming and SIM management	Device HTTPS/MQTT -> station-ingest
LTE/4G	Webcams and high-bandwidth stations	Supports images/video and frequent telemetry	Higher power/cost	HTTPS/MQTT/S3 upload
Wi-Fi/Ethernet	Club house, school, landing field with internet	Low operating cost	Not suitable for remote launch without infrastructure	HTTPS/MQTT
BLE	Commissioning and local service	Simple setup from mobile app	Short range, not for telemetry	Mobile app -> station config

6.2 LoRaWAN Integration

LoRaWAN is used for low-bandwidth telemetry, not webcam imagery. The architecture uses a network-server adapter rather than connecting business logic directly to one provider. This lets the platform support The Things Stack, private gateways, or future commercial network servers.

Station -> LoRaWAN Gateway -> Network Server -> MQTT/Webhook -> station-ingest-service

Topic example:

v3/{application_id}@{tenant}/devices/{device_id}/up

Adapter output:

canonical.station.uplink.received

Adaptive Data Rate should be enabled for stationary stations after placement is stable. For mobile/portable event nodes, ADR may be disabled or tuned because link conditions change more often.

6.3 Cellular Integration

Cellular stations connect directly to the platform through HTTPS or MQTT over TLS. LTE-M/NB-IoT is preferred for telemetry-only devices where supported; standard LTE is required for cameras or high-frequency payloads. SIM inventory, roaming, data caps, and operator coverage become part of the station operations system.

```
POST /v1/stations/{station_id}/uplinks
Authorization: Device-Signature ...
Content-Type: application/cbor or application/json
```

```
{
  "seq": 91221,
  "observed_at": "2026-06-05T12:01:00Z",
  "interval_s": 60,
```

```

    "measurements": {"wind_speed_avg_ms": 4.8, "wind_gust_max_ms": 8.2}
}

```

7. Edge Firmware Architecture

7.1 Firmware State Machine

BOOT

```

-> verify firmware signature
-> load config
-> initialize sensors
-> sample loop
-> aggregate window
-> local QC
-> persist payload
-> transmit
-> wait/sleep
-> repeat

```

Failure branches:

```

sensor_error -> mark metric invalid -> transmit health
connectivity_error -> queue payload -> retry/backoff
low_battery -> reduce frequency -> send power warning
firmware_update -> download -> verify -> install -> rollback on failure

```

7.2 Firmware Modules

Module	Responsibilities	Test requirements
sensor_driver_*	Read raw sensor data, expose normalized units	hardware-in-loop sensor simulation
aggregator	Compute vector wind average, gust max, variance	golden vector tests for direction wrap-around
local_qc	Detect impossible values, stuck sensors, missing sensor	unit tests for each QC rule
payload_encoder	Encode JSON/CBOR/protobuf payload	schema compatibility tests
radio_manager	LoRaWAN/cellular session and retry logic	network outage tests
power_manager	Battery state, sleep profile, low-power mode	battery discharge simulation
config_manager	Remote config and local safe defaults	rollback and invalid config tests
ota_updater	Signed firmware update and rollback	signature failure tests

7.3 Local Quality Control Rules

```

if wind_speed_avg_ms < 0 or wind_speed_avg_ms > 70: invalid('wind_range')
if wind_gust_max_ms < wind_speed_avg_ms: invalid('gust_less_than_average')
if abs(delta(direction_deg)) > 180: use_vector_average()
if repeated_same_value(sensor, duration_min=30): warn('possible_stuck_sensor')
if battery_voltage < threshold_critical: enter_low_power_mode()
if observed_at older than last_observed_at: reject_or_resequence()

```

8. Payloads and Protocols

8.1 Canonical Observation Payload

```

{
  "station_id": "stn_vaga_launch_01",
  "device_id": "dev_70b3d57ed006ab12",
  "seq": 812903,
  "firmware_version": "1.4.2",
  "observed_at": "2026-06-05T13:02:00Z",
  "transmitted_at": "2026-06-05T13:02:04Z",
  "interval_s": 60,

```

```

"location": {"lat": 61.875, "lon": 9.095, "alt_m": 950},
"measurements": {
  "wind_speed_avg_ms": 4.8,
  "wind_gust_max_ms": 8.2,
  "wind_direction_deg": 315,
  "temperature_c": 16.4,
  "relative_humidity_pct": 56,
  "pressure_hpa": 912.3,
  "battery_voltage_v": 12.4,
  "solar_current_a": 0.8
},
"radio": {"rssi_dbm": -96, "snr_db": 7.2, "network": "lorawan"},
"quality": {"edge_flags": []}
}

```

8.2 Event Topics

Topic / event	Producer	Consumers	Purpose
station.uplink.raw	adapter services	normalizer, audit storage	Raw payload preservation
station.observation.normalized	station-ingest	QC service, Timescale writer	Canonical observation stream
station.observation.qualified	QC service	weather engine, flyability, digital twin	Use in forecast correction and app display
station.health.changed	health service	ops alerts, club dashboards	Maintenance notifications
station.config.updated	station admin API	device config service	Remote station configuration
station.offline.detected	health service	notification service	Notify owner/club
forecast.reality.delta.updated	weather engine	mobile API, AI briefing, digital twin	Forecast-vs-reality signal

9. Cloud Ingestion Architecture

9.1 Services

Service	Owned data	Responsibilities	Scale trigger
station-adapter-lorawan	raw LoRaWAN uplinks	Validate network-server signatures, decode payloads, map device IDs	uplinks/sec
station-adapter-cellular	direct device uplinks	Device auth, schema validation, idempotency	requests/sec
station-normalizer	canonical observations	Unit conversion, field mapping, timestamp normalization	queue lag
station-qc-service	quality flags	Range checks, spike checks, station drift, duplicate detection	observation volume
station-health-service	status snapshots	Online/offline detection, battery and signal trend analysis	station count
station-config-service	desired config	Remote configuration, OTA rollout groups	device count
station-observation-writer	Timescale rows	Batch write qualified observations	write latency
station-provenance-service	lineage	Source attribution, licensing, audit evidence	audit volume

9.2 Ingestion Sequence

1. Device sends uplink to LoRaWAN network server or direct HTTPS endpoint.
2. Adapter verifies source, parses transport metadata, and stores raw payload pointer.
3. Normalizer maps vendor payload to canonical schema and canonical units.
4. QC service applies hard validation, suspicious flags, and station-specific calibration.
5. Qualified observation is written to TimescaleDB.
6. Weather engine updates station-to-forecast delta and site nowcast state.
7. Alert engine evaluates stale station, gust exceedance, and sudden wind shift.
8. Mobile/Web API exposes live observations with confidence and freshness labels.

9.3 Failure Handling

Failure	Detection	Behavior
Duplicate payload	station_id + seq + observed_at unique key	Ignore duplicate but record duplicate count
Late payload	observed_at older than freshness window	Store for ML/history but do not use for live safety decisions
Bad timestamp	clock skew > tolerance	Use server receive time; flag timestamp_suspect
Missing gust	schema validation or sensor absent	Use average wind only; lower confidence; no gust-driven GO decision
Sensor stuck	same value too long	Flag station degraded; remove from flyability hard blockers after threshold
Station offline	no qualified uplink for configured interval	Show stale status; notify owner; do not present as live
Battery critical	battery below threshold	Reduce transmit frequency; send urgent maintenance alert

10. Database Extensions

The database extensions align with Volume 03. Station metadata is relational/PostGIS. High-volume observations are TimescaleDB hypertables. Raw payloads and images are object storage with database pointers.

```
CREATE TABLE stations (  
  id UUID PRIMARY KEY,  
  site_id UUID REFERENCES flying_sites(id),  
  owner_org_id UUID,  
  station_code TEXT UNIQUE NOT NULL,  
  station_type TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'planned',  
  location GEOGRAPHY(POINT,4326) NOT NULL,  
  altitude_m NUMERIC,  
  mast_height_m NUMERIC,  
  exposure_class TEXT,  
  installed_at TIMESTAMPTZ,  
  last_seen_at TIMESTAMPTZ,  
  firmware_version TEXT,  
  connectivity_primary TEXT,  
  connectivity_fallback TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE TABLE station_devices (  
  id UUID PRIMARY KEY,  
  station_id UUID REFERENCES stations(id),  
  hardware_serial TEXT UNIQUE NOT NULL,  
  device_eui TEXT UNIQUE,  
  imei TEXT UNIQUE,  
  public_key TEXT,  
  active BOOLEAN DEFAULT TRUE,  
  commissioned_at TIMESTAMPTZ,  
  revoked_at TIMESTAMPTZ  
);
```

```
CREATE TABLE station_observations (  
  station_id UUID NOT NULL REFERENCES stations(id),  
  observed_at TIMESTAMPTZ NOT NULL,  
  received_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  seq BIGINT,  
  wind_speed_avg_ms NUMERIC,  
  wind_gust_max_ms NUMERIC,  
  wind_direction_deg NUMERIC,  
  wind_direction_stddev_deg NUMERIC,  
  temperature_c NUMERIC,  
  relative_humidity_pct NUMERIC,
```



```

    pressure_hpa NUMERIC,
    rain_rate_mm_h NUMERIC,
    solar_irradiance_wm2 NUMERIC,
    battery_voltage_v NUMERIC,
    radio_rssi_dbm NUMERIC,
    radio_snr_db NUMERIC,
    qc_status TEXT NOT NULL DEFAULT 'pending',
    qc_flags TEXT[] DEFAULT '{}',
    PRIMARY KEY (station_id, observed_at)
);
-- SELECT create_hypertable('station_observations','observed_at');

CREATE INDEX idx_station_observations_recent
ON station_observations (station_id, observed_at DESC);

CREATE TABLE station_health_snapshots (
    station_id UUID REFERENCES stations(id),
    checked_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    health_state TEXT NOT NULL,
    battery_state TEXT,
    connectivity_state TEXT,
    sensor_state TEXT,
    stale_minutes INT,
    active_alerts TEXT[],
    PRIMARY KEY (station_id, checked_at)
);

CREATE TABLE station_calibrations (
    id UUID PRIMARY KEY,
    station_id UUID REFERENCES stations(id),
    sensor_name TEXT NOT NULL,
    calibration_type TEXT NOT NULL,
    coefficients JSONB NOT NULL,
    valid_from TIMESTAMPTZ NOT NULL,
    valid_to TIMESTAMPTZ,
    approved_by UUID,
    evidence_uri TEXT
);

CREATE TABLE station_raw_payloads (
    id UUID PRIMARY KEY,
    station_id UUID REFERENCES stations(id),
    received_at TIMESTAMPTZ DEFAULT now(),
    transport TEXT NOT NULL,
    payload_uri TEXT NOT NULL,
    payload_hash TEXT NOT NULL,
    decoded BOOLEAN DEFAULT FALSE
);

```

11. APIs and Service Contracts

11.1 Public/Partner Station APIs

```

GET /v1/sites/{site_id}/stations
GET /v1/stations/{station_id}
GET /v1/stations/{station_id}/observations?from=&to=&resolution=1m
GET /v1/stations/{station_id}/health
GET /v1/stations/{station_id}/forecast-delta
POST /v1/stations/{station_id}/reports/maintenance
POST /v1/partner-stations/uplinks

```

11.2 Owner/Admin APIs

```

POST /v1/admin/stations
PATCH /v1/admin/stations/{station_id}

```

```

POST /v1/admin/stations/{station_id}/commission
POST /v1/admin/stations/{station_id}/config
POST /v1/admin/stations/{station_id}/calibrations
POST /v1/admin/stations/{station_id}/ota-rollout
POST /v1/admin/stations/{station_id}/decommission

```

11.3 Example Observation Response

```

{
  "station_id": "stn_vaga_launch_01",
  "freshness": "live",
  "source_confidence": 0.94,
  "observations": [
    {
      "observed_at": "2026-06-05T13:02:00Z",
      "wind_speed_avg_ms": 4.8,
      "wind_gust_max_ms": 8.2,
      "wind_direction_deg": 315,
      "qc_status": "qualified",
      "qc_flags": []
    }
  ]
}

```

12. Quality Control and Calibration

12.1 QC Pipeline

```

raw_payload
-> schema_validation
-> unit_normalization
-> physical_range_checks
-> temporal_consistency_checks
-> station_calibration
-> neighbor_station_cross_check
-> forecast_plausibility_check
-> qualified/degraded/rejected

```

12.2 QC Flags

Flag	Severity	Meaning	Product behavior
range_invalid	reject	Value physically impossible	Do not show/use
timestamp_suspect	degraded	Device time appears wrong	Show if corrected; not for alert decisions
stale	degraded	No recent observation	Show stale badge; no live safety decision
sensor_stuck	degraded	Sensor value unchanged too long	Remove from hard safety blockers
gust_suspect	degraded	Gust below average or impossible spike	Do not use gust for scoring
battery_low	warning	Power risk	Notify owner
station_moved	warning	GNSS/location changed	Require admin review
calibration_expired	warning	Calibration evidence outdated	Lower source confidence

12.3 Calibration Model

- Every permanent station must have installation photos, mast height, exposure class, and location confirmed during commissioning.
- Wind sensors require a calibration record or manufacturer calibration metadata where available.
- Calibration coefficients are applied by station-qc-service, not by client apps.
- When calibration changes, historical raw payloads are preserved; corrected views can be recomputed for ML training.
- Calibration updates require RBAC permission station.calibration.write and audit logging.

13. Station Health and Operations

13.1 Health Score

```
health_score =  
    freshness_score * 0.30 +  
    battery_score * 0.20 +  
    connectivity_score * 0.20 +  
    sensor_score * 0.20 +  
    calibration_score * 0.10
```

States:

GOOD >= 85

DEGRADED 60-84

MAINTENANCE_REQUIRED 30-59

OFFLINE < 30

13.2 Operational Alerts

Alert	Condition	Recipient
station_offline	No qualified uplink for N intervals	station owner, club admin, ops
battery_critical	Battery below critical threshold	owner, ops
sensor_fault	Repeated invalid/stuck sensor flags	owner, ops
high_gust_live	Gust exceeds site safety threshold	pilots subscribed to site
wind_shift_live	Direction change makes launch unsafe	pilots on site, club admin
maintenance_due	Calibration/service interval reached	owner, club admin

13.3 Operations Dashboard

- Fleet map with station status color: good, degraded, maintenance, offline.
- Per-station timeline: observations, battery, signal, QC flags, alerts, firmware version.
- Owner workflows: acknowledge alert, create maintenance ticket, upload service photos, update calibration.
- Ops workflows: OTA rollout groups, config drift check, stale station report, SIM data usage report.
- Club view: simplified status, live wind, forecast delta, and maintenance tasks.

14. Weather Engine Integration

14.1 Forecast-Versus-Reality Delta

Every qualified station observation is matched to the nearest model forecast point and model valid time. The weather engine computes forecast error samples that feed site digital twins and real-time confidence labels.

```
forecast_error_sample = {  
    site_id, station_id, model_name, model_run_id, forecast_valid_at, observed_at,  
    forecast_wind_speed_ms, observed_wind_speed_ms, error_wind_speed_ms,  
    forecast_wind_direction_deg, observed_wind_direction_deg, error_direction_deg,  
    forecast_gust_ms, observed_gust_ms, error_gust_ms,  
    terrain_context, exposure_class, qc_status  
}
```

14.2 Flyability Integration

Condition	Scoring behavior
Fresh station confirms forecast	Increase confidence and display live validation
Station shows stronger wind than forecast	Use station-adjusted wind for launch score and warn model under-forecasting
Station stale	Do not use as live data; reduce confidence if no alternate reality source
Station degraded	Display with warning; do not trigger hard GO from degraded data
Live gust exceeds site limit	Hard blocker: status NO-GO for affected launch and pilot profiles
Wind direction outside launch sector	Direction blocker or MAYBE depending playbook and pilot level

14.3 Digital Twin Integration

The digital twin learns station-specific biases by weather regime: synoptic direction, stability, time of day, season, lapse rate, and terrain exposure. Station observations are the primary ground-truth for wind correction; pilot reports and track-derived drift are secondary evidence.

15. Mobile and Web UX

15.1 Pilot-Facing UI

UI component	Content	Safety rule
Live wind card	Average, gust, direction, timestamp, source confidence	Always show freshness and station status
Forecast vs reality badge	Model wind vs observed wind delta	High disagreement lowers GO confidence
Station detail screen	Chart, health, battery, owner, elevation, mast height	Do not hide degraded/stale state
Map icon	Station marker colored by status and wind safety	NO-GO colors must be visually distinct
Alert panel	Live gust, wind shift, station offline	Critical alerts must be push-capable
Owner dashboard	Maintenance tasks, battery, signal, QC flags	No public display of sensitive owner contact

15.2 AI Briefing Rules

- AI may cite station data only if freshness and QC status are included in the context.
- AI must not call stale station data live.
- AI must mention model disagreement when station data contradicts the forecast.
- AI must prefer hard safety blockers from station observations over generic forecast optimism.
- AI must say insufficient live data when station status is offline/degraded and no alternate evidence exists.

16. Security and Privacy

16.1 Device Identity

- Every station device receives a unique hardware identity and platform device record.
- Direct cellular devices authenticate with signed requests or mutual TLS where feasible.
- LoRaWAN devices are authenticated by network-server credentials plus platform-side device mapping.
- Device credentials are revocable without deleting historical station records.
- Commissioning requires admin/owner authorization and audit logging.

16.2 Threat Controls

Threat	Control
Fake station uplinks	Device signatures, network-server signature validation, device allowlist
Replay attacks	Sequence number, observed_at window, idempotency key
Payload tampering	TLS, payload hash, raw payload archive
Malicious public reports overriding instruments	Separate community report confidence from station confidence
OTA compromise	Signed firmware, rollout groups, rollback, audit
Owner privacy leak	Separate owner contact details from public station metadata
Station location abuse	Public location allowed only for weather stations; private test stations can be hidden

17. Installation Playbook

17.1 Site Survey

1. Identify launch, landing, ridge and valley measurement objectives.
2. Check radio coverage options: LoRaWAN gateway reach, cellular signal, Wi-Fi/Ethernet availability.
3. Choose mast location with safe access, representative wind exposure, minimal rotor distortion, and low vandalism risk.
4. Document GPS location, altitude, mast height, photos in four directions, and obstruction notes.
5. Estimate solar exposure, snow load, wind loading, and maintenance access route.
6. Confirm club/site owner permission and data publication rules.

17.2 Commissioning Checklist

- Create station record and assign owner organization.
- Register device identity and connectivity profile.
- Upload installation photos and mast metadata.
- Run sensor self-test and wind-direction orientation check.
- Verify first uplink and canonical observation storage.
- Run 24-hour burn-in before station can influence flyability hard blockers.
- Approve station for live display after QC pass and owner sign-off.

18. Manufacturing and Supply Chain

18.1 Build Strategy

Phase	Approach	Reason
MVP	Assemble from off-the-shelf sensors and industrial enclosures	Fast validation, low tooling cost
Pilot commercial	Standardize BOM and mounting kit for 20-100 stations	Repeatability and service training
Scale	Partner with hardware manufacturer for enclosure, PCB, certification path	Quality and cost control
Enterprise	Offer rugged premium station and service contract	Higher ARPU from clubs, schools, events, resorts

18.2 Bill of Materials Categories

- Wind sensor, environmental sensor, optional rain/solar sensors, optional camera.
- MCU/controller, radio module, SIM/eSIM module, antenna and cable set.
- Solar panel, charge controller, battery, fuses, power monitoring.
- IP-rated enclosure, cable glands, mast mounting hardware, grounding accessories.
- QR code label, serial label, field service kit, spare sensor kit.

19. Commercial Model

19.1 Revenue Lines

Line	Customer	Pricing logic	Included services
Station hardware sale	Clubs, schools, sites	Margin on hardware kit	Device, enclosure, setup guide
Station subscription	Station owners	Monthly/annual per station	Cloud ingestion, dashboards, alerts, storage
Managed station service	Clubs/schools/events	Higher monthly fee	Monitoring, maintenance reminders, support
Event station rental	Competitions/festivals	Per event/week	Portable nodes, dashboard, live briefing
Data API partner	Apps/hardware makers	API calls or station count	Station observations and forecast deltas

19.2 Tier Entitlements

- Free users can view delayed/basic station data where allowed by station owner.
- Pro users can view live station charts and forecast-vs-reality deltas for favorite sites.
- Pro+ users get model-disagreement explanations, station-driven AI briefings, and station alert automation.
- Club tier gets station owner dashboard, maintenance tickets, member alerts, and playbook integration.
- School/event tiers get multi-station operational dashboards and briefing export.

20. MVP Scope and Roadmap

20.1 MVP

- Integrate third-party station ingestion through HTTP/MQTT adapter.
- Support one first-party prototype station with wind, temperature, humidity, pressure, battery, and LoRaWAN/cellular path.
- Implement station tables, observation hypertable, health snapshots, and QC flags.
- Show live/stale/degraded station card on launch detail screen.
- Compute forecast-vs-reality delta for wind speed and direction.
- Trigger live gust and station-offline alerts.
- Provide owner dashboard for station health and maintenance notes.

20.2 V1

- LoRaWAN network-server adapters for The Things Stack and custom webhook sources.
- LTE direct device API with signed payloads.
- Station commissioning workflow in mobile app.
- Calibration records and evidence upload.
- Station health score and fleet dashboard.
- Integration with digital twin feature store.

20.3 V2/V3

- Premium webcam station with visibility/cloud analysis.
- OTA firmware management and staged rollout.
- Portable event station rental package.
- Neighbor station cross-validation and automatic sensor-drift detection.
- Hardware marketplace and certified partner program.
- Global station network analytics and public station map.

21. Acceptance Criteria

Area	Acceptance criteria
Observation ingestion	95% of valid test uplinks appear as qualified observations within 10 seconds under normal load
Duplicate handling	Repeated seq/observed_at payloads do not create duplicate observation rows
Freshness labeling	Station data older than configured window is always displayed as stale
Safety behavior	No GO recommendation may be upgraded by stale or degraded station data
Battery alerts	Battery critical event creates owner alert and dashboard state
Forecast delta	Qualified station observations produce forecast error sample within one processing cycle
Offline mode	Offline station produces health state and owner notification after threshold
Security	Unregistered devices and invalid signatures are rejected and audited

Mobile UX	Pilot can see wind, gust, direction, timestamp, QC state, and station source on launch screen
Ops dashboard	Operator can filter station fleet by offline, degraded, low battery, and firmware version

22. Engineering Backlog

Epic	Priority	Tasks
Station ingestion MVP	P0	Canonical payload schema, HTTP endpoint, MQTT adapter, raw payload archive
Station database	P0	stations, devices, observations hypertable, health snapshots, calibration records
QC engine	P0	range checks, timestamp checks, stale detection, gust sanity, sensor stuck detection
Launch UI integration	P0	live wind card, stale/degraded badges, station chart
Weather engine delta	P0	match station obs to forecast cube, compute wind/direction/gust error
Station alerts	P1	gust threshold, wind shift, offline, battery low
Commissioning app	P1	QR scan, site assignment, photos, mast height, first uplink test
LoRaWAN adapter	P1	The Things Stack webhook/MQTT parser and device mapping
Cellular device auth	P1	signed payloads, key rotation, revocation
Fleet dashboard	P1	health map, maintenance tickets, firmware/config status
OTA firmware	P2	signed artifacts, rollout groups, rollback
Premium camera station	P2	image upload, blur/darkness detection, cloud/visibility models

23. References

The following references informed the architecture. Product implementation must verify licenses, pricing, and technical requirements again before procurement or launch.

- LoRa Alliance - LoRaWAN specification overview and Adaptive Data Rate: <https://loro-alliance.org/about-lorawan-old/>
- OASIS MQTT Version 5.0 standard: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- The Things Stack MQTT integration documentation: <https://www.thethingsindustries.com/docs/integrations/other-integrations/mqtt/>
- The Things Stack Webhooks documentation: <https://www.thethingsindustries.com/docs/integrations/webhooks/>
- The Things Network LoRaWAN overview and ADR guidance: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> and <https://www.thethingsnetwork.org/docs/lorawan/adaptive-data-rate/>
- GSMA Mobile IoT deployment and LTE-M/NB-IoT information: <https://www.gsma.com/solutions-and-impact/technologies/internet-of-things/deployment-map/>